

BẢN MÔ TẢ CHI TIẾT DỰ ÁN

Đề tài: Ứng dụng chữ ký số RSA để xác thực nội dung văn bản – có kiểm tra sửa đổi

1. Giới thiệu đề tài

1.1. Tên đề tài

Ứng dụng chữ ký số RSA để xác thực nội dung văn bản – có kiểm tra sửa đổi

1.2. Mục tiêu của đề tài

Đề tài xây dựng một ứng dụng cho phép người dùng tạo chữ ký số RSA cho nội dung văn bản, sau đó sử dụng chữ ký đó để xác thực lại văn bản. Hệ thống có khả năng phát hiện văn bản đã bị thay đổi sau khi ký.

Mục tiêu chính của hệ thống gồm:

- Tạo cặp khóa RSA gồm khóa riêng tư và khóa công khai.
- Sử dụng khóa riêng tư để ký nội dung văn bản.
- Sử dụng khóa công khai để xác thực chữ ký.
- Kiểm tra tính toàn vẹn của văn bản.
- Phát hiện văn bản bị sửa đổi sau khi đã ký.
- Hỗ trợ người dùng lưu khóa, chữ ký và kết quả xác thực.

1.3. Ý nghĩa của đề tài

Trong thực tế, các văn bản điện tử như hợp đồng, đơn xác nhận, biên bản, thông báo hoặc tài liệu nội bộ có thể bị chỉnh sửa sau khi gửi đi. Nếu không có cơ chế xác thực, người nhận khó biết được văn bản có còn nguyên vẹn hay không.

Chữ ký số RSA giúp giải quyết vấn đề này bằng cách:

- Xác nhận văn bản do đúng người ký tạo ra.
 - Đảm bảo nội dung văn bản không bị thay đổi.
 - Hạn chế việc giả mạo chữ ký.
 - Tăng tính tin cậy khi trao đổi văn bản điện tử.
-

2. Phạm vi của dự án

2.1. Phạm vi chức năng

Hệ thống tập trung vào các chức năng chính sau:

1. Quản lý khóa RSA.
2. Ký số nội dung văn bản.
3. Xác thực chữ ký số.
4. Kiểm tra văn bản có bị sửa đổi hay không.
5. Lưu trữ khóa, chữ ký, văn bản và lịch sử xác thực.
6. Hiển thị kết quả rõ ràng cho người dùng.

2.2. Đối tượng dữ liệu xử lý

Hệ thống có thể xử lý các loại dữ liệu sau:

- Văn bản nhập trực tiếp từ bàn phím.
- File văn bản `.txt`.
- Có thể mở rộng sang file `.pdf`, `.docx`.

Trong phạm vi cơ bản của đề tài, nên ưu tiên xử lý:

- Nội dung text nhập trực tiếp.
- File `.txt`.

Sau khi hoàn thành phần lõi, có thể mở rộng sang PDF.

2.3. Đối tượng sử dụng

Người dùng của hệ thống có thể là:

- Sinh viên hoặc giảng viên cần kiểm tra tính toàn vẹn tài liệu.
- Người gửi văn bản cần ký số trước khi gửi.
- Người nhận văn bản cần xác thực chữ ký.
- Người quản trị hệ thống nếu có chức năng quản lý người dùng.

3. Cơ sở nghiệp vụ của hệ thống

3.1. Bản chất của chữ ký số RSA

Chữ ký số RSA không phải là việc mã hóa toàn bộ văn bản bằng khóa riêng tư. Quy trình đúng thường là:

1. Văn bản được đưa qua hàm băm SHA-256.
2. Hệ thống tạo ra mã băm đại diện cho nội dung văn bản.
3. Mã băm được ký bằng khóa riêng tư RSA.

4. Kết quả ký chính là chữ ký số.

Khi xác thực:

1. Hệ thống băm lại văn bản cần kiểm tra.
2. Hệ thống dùng khóa công khai để kiểm tra chữ ký.
3. Nếu chữ ký hợp lệ và mã băm khớp, văn bản chưa bị sửa đổi.
4. Nếu văn bản bị sửa dù chỉ một ký tự, mã băm sẽ thay đổi và xác thực thất bại.

3.2. Vai trò của khóa riêng tư

Khóa riêng tư dùng để tạo chữ ký số.

Quy tắc nghiệp vụ:

- Chỉ chủ sở hữu mới được giữ khóa riêng tư.
- Không chia sẻ khóa riêng tư cho người khác.
- Nếu khóa riêng tư bị lộ, chữ ký có thể bị giả mạo.
- Hệ thống cần lưu khóa riêng tư ở nơi an toàn.

3.3. Vai trò của khóa công khai

Khóa công khai dùng để xác thực chữ ký.

Quy tắc nghiệp vụ:

- Khóa công khai có thể chia sẻ cho người khác.
- Người nhận văn bản dùng khóa công khai để kiểm tra chữ ký.
- Khóa công khai không dùng để ký văn bản.
- Nếu dùng sai khóa công khai, xác thực sẽ thất bại.

3.4. Vai trò của hàm băm SHA-256

SHA-256 dùng để tạo giá trị đại diện cho nội dung văn bản.

Ví dụ:

Văn bản gốc:

Tôi đồng ý với nội dung văn bản này.

Văn bản bị sửa:

Tôi không đồng ý với nội dung văn bản này.

Chỉ cần thay đổi một từ, mã băm SHA-256 sẽ thay đổi hoàn toàn. Nhờ đó, hệ thống có thể phát hiện văn bản đã bị sửa.

4. Mô hình tổng quan của hệ thống

4.1. Mô hình xử lý chính

Hệ thống gồm ba nhóm chức năng chính:

Quản lý khóa RSA
↓
Ký số văn bản
↓
Xác thực và kiểm tra sửa đổi

4.2. Mô hình luồng ký văn bản

Người dùng nhập/chọn văn bản
↓
Hệ thống chuẩn hóa nội dung văn bản
↓
Hệ thống băm văn bản bằng SHA-256
↓
Người dùng chọn khóa riêng tư
↓
Hệ thống ký mã băm bằng RSA
↓
Sinh chữ ký số
↓
Lưu hoặc hiển thị chữ ký số

4.3. Mô hình luồng xác thực văn bản

Người dùng nhập/chọn văn bản cần kiểm tra
↓
Người dùng nhập/chọn chữ ký số
↓
Người dùng chọn khóa công khai
↓
Hệ thống băm lại văn bản
↓
Hệ thống xác thực chữ ký bằng RSA
↓
Trả về kết quả hợp lệ hoặc không hợp lệ

5. Các chức năng chính của hệ thống

5.1. Chức năng tạo cặp khóa RSA

Mục đích

Cho phép người dùng tạo một cặp khóa RSA gồm:

- Khóa riêng tư.
- Khóa công khai.

Đầu vào

Người dùng nhập:

- Tên khóa.
- Tên chủ sở hữu.
- Độ dài khóa, ví dụ 2048 bit hoặc 3072 bit.

Xử lý nghiệp vụ

Hệ thống thực hiện:

1. Kiểm tra tên khóa có bị trống không.
2. Kiểm tra tên chủ sở hữu có bị trống không.
3. Kiểm tra tên khóa đã tồn tại chưa.
4. Sinh cặp khóa RSA.
5. Lưu khóa riêng tư vào file.
6. Lưu khóa công khai vào file.
7. Lưu thông tin khóa vào hệ thống.

Đầu ra

Hệ thống trả về:

- Thông báo tạo khóa thành công.
- Mã khóa hoặc tên khóa.
- Đường dẫn lưu khóa riêng tư.
- Đường dẫn lưu khóa công khai.

Quy tắc nghiệp vụ

- Tên khóa không được để trống.
- Tên khóa không được trùng.
- Khóa riêng tư phải được lưu tách biệt với khóa công khai.
- Khóa riêng tư không nên hiển thị công khai trên giao diện.
- Khóa công khai có thể cho phép xem hoặc tải xuống.

5.2. Chức năng xem danh sách khóa

Mục đích

Cho phép người dùng xem các khóa đã được tạo trong hệ thống.

Đầu vào

Không bắt buộc nhập dữ liệu.

Xử lý nghiệp vụ

Hệ thống đọc danh sách khóa đã lưu.

Mỗi khóa có thể gồm:

- Mã khóa.
- Tên khóa.
- Chủ sở hữu.
- Ngày tạo.
- Trạng thái khóa.

Đầu ra

Hiển thị danh sách khóa cho người dùng.

Ví dụ:

1. Key_Minh - Chủ sở hữu: Nguyễn Văn Minh - Ngày tạo: 05/06/2026
 2. Key_Test - Chủ sở hữu: Demo User - Ngày tạo: 05/06/2026

Quy tắc nghiệp vụ

- Nếu chưa có khóa nào, hiển thị thông báo: "Chưa có khóa nào".
- Không hiển thị toàn bộ khóa riêng tư trong danh sách.
- Chỉ hiển thị thông tin nhận diện khóa.

5.3. Chức năng xem khóa công khai

Mục đích

Cho phép người dùng xem nội dung khóa công khai để chia sẻ cho người nhận văn bản.

Đầu vào

Người dùng chọn một khóa trong danh sách.

Xử lý nghiệp vụ

Hệ thống:

1. Kiểm tra khóa có tồn tại không.
2. Tìm file `public_key.pem`.
3. Đọc nội dung khóa công khai.
4. Hiển thị cho người dùng.

Đầu ra

Hiển thị nội dung khóa công khai ở định dạng PEM.

Ví dụ:

```
-----BEGIN PUBLIC KEY-----  
...  
-----END PUBLIC KEY-----
```

Quy tắc nghiệp vụ

- Chỉ cho phép xem khóa công khai.
- Nếu không tìm thấy file khóa, báo lỗi.
- Không yêu cầu mật khẩu để xem khóa công khai trong mô hình cơ bản.

5.4. Chức năng lưu khóa

Mục đích

Cho phép hệ thống lưu lại khóa để sử dụng cho các lần ký và xác thực sau.

Dữ liệu cần lưu

Mỗi cặp khóa nên được lưu trong một thư mục riêng.

Ví dụ:

```
keys/  
|  
└─ Key_Minh/
```

```
| | | private_key.pem
| | | public_key.pem
| | | metadata.json
| |
| | --- Key_Test/
| | | private_key.pem
| | | public_key.pem
| | | metadata.json
```

File `metadata.json` có thể chứa:

```
{
  "key_id": "Key_Minh",
  "owner_name": "Nguyễn Văn Minh",
  "created_at": "2026-06-05 10:30:00",
  "algorithm": "RSA",
  "key_size": 2048
}
```

Quy tắc nghiệp vụ

- Không ghi đè khóa nếu tên khóa đã tồn tại.
- Nếu muốn tạo lại khóa cùng tên, cần yêu cầu xác nhận.
- Khóa riêng tư cần được bảo vệ tốt hơn khóa công khai.

5.5. Chức năng xóa khóa

Mục đích

Cho phép người dùng xóa cặp khóa không còn sử dụng.

Đầu vào

Người dùng chọn khóa cần xóa.

Xử lý nghiệp vụ

Hệ thống:

1. Hiển thị thông tin khóa cần xóa.
2. Yêu cầu người dùng xác nhận.
3. Nếu người dùng đồng ý, xóa thư mục chứa khóa.
4. Cập nhật danh sách khóa.

Đầu ra

Thông báo:

Đã xóa khóa thành công.

hoặc:

Đã hủy thao tác xóa khóa.

Quy tắc nghiệp vụ

- Xóa khóa là thao tác nguy hiểm.
- Cần xác nhận trước khi xóa.
- Nếu khóa đã dùng để ký văn bản trước đó, việc xóa khóa riêng tư không làm mất khả năng xác thực nếu vẫn còn khóa công khai.
- Nếu xóa cả khóa công khai, các văn bản cũ sẽ khó xác thực lại.

5.6. Chức năng nhập văn bản cần ký

Mục đích

Cho phép người dùng đưa nội dung văn bản vào hệ thống để ký số.

Hình thức nhập

Có thể hỗ trợ hai cách:

1. Nhập trực tiếp vào ô văn bản.
2. Tải file `.txt` từ máy tính.

Đầu vào

- Nội dung văn bản.
- Hoặc file `.txt`.

Xử lý nghiệp vụ

Hệ thống:

1. Kiểm tra văn bản có bị trống không.
2. Nếu là file, kiểm tra định dạng file.
3. Đọc nội dung file.
4. Chuẩn hóa nội dung trước khi băm.

5. Chuyển nội dung sang bytes để xử lý ký số.

Quy tắc nghiệp vụ

- Không cho phép ký văn bản rỗng.
- Nếu file không đọc được, báo lỗi.
- Nếu file không đúng định dạng, báo lỗi.
- Nội dung dùng để ký phải giống nội dung dùng để xác thực sau này.

5.7. Chức năng tạo mã băm SHA-256

Mục đích

Tạo mã băm đại diện cho nội dung văn bản.

Đầu vào

Nội dung văn bản.

Xử lý nghiệp vụ

Hệ thống:

1. Nhận nội dung văn bản.
2. Chuyển nội dung sang dạng bytes.
3. Dùng SHA-256 để tạo mã băm.
4. Trả về chuỗi hash dạng hexadecimal.

Đầu ra

Ví dụ:

SHA-256 :
a3f5c9e8...

Quy tắc nghiệp vụ

- Cùng một nội dung phải cho ra cùng một mã băm.
- Chỉ cần sửa một ký tự, mã băm phải thay đổi.
- Mã băm có thể hiển thị trên giao diện để minh họa tính toàn vẹn.

5.8. Chức năng ký văn bản

Mục đích

Tạo chữ ký số RSA cho văn bản bằng khóa riêng tư.

Đầu vào

Người dùng cung cấp:

- Nội dung văn bản.
- Khóa riêng tư.
- Tên người ký hoặc mã khóa.

Xử lý nghiệp vụ

Hệ thống:

1. Kiểm tra văn bản có hợp lệ không.
2. Kiểm tra khóa riêng tư có tồn tại không.
3. Tạo mã băm SHA-256 của văn bản.
4. Dùng khóa riêng tư RSA để ký mã băm.
5. Chuyển chữ ký sang dạng Base64 để dễ lưu trữ và hiển thị.
6. Trả về chữ ký số.

Đầu ra

Hệ thống hiển thị:

- Nội dung văn bản.
- Mã băm SHA-256.
- Chữ ký số.
- Tên khóa đã dùng.
- Thời gian ký.

Ví dụ kết quả:

Ký văn bản thành công.

Người ký: Nguyễn Văn Minh

Khóa sử dụng: Key_Minh

Thuật toán: RSA + SHA-256

Thời gian ký: 05/06/2026 10:30:00

Quy tắc nghiệp vụ

- Chỉ khóa riêng tư mới được dùng để ký.

- Không dùng khóa công khai để ký.
- Nếu khóa riêng tư sai định dạng, báo lỗi.
- Nếu văn bản bị thay đổi sau khi ký, chữ ký cũ sẽ không còn hợp lệ.

5.9. Chức năng lưu chữ ký số

Mục đích

Lưu chữ ký số để phục vụ xác thực sau này.

Dữ liệu cần lưu

Có thể lưu chữ ký trong file `.sig` hoặc `.txt`.

Ví dụ:

```
signatures/  
|  
├─ document_001_signature.sig  
└─ document_002_signature.sig
```

Nội dung file chữ ký có thể gồm:

```
{  
  "document_name": "vanban1.txt",  
  "signature": "Base64SignatureHere",  
  "hash_algorithm": "SHA-256",  
  "signature_algorithm": "RSA",  
  "key_id": "Key_Minh",  
  "signed_at": "2026-06-05 10:30:00"  
}
```

Quy tắc nghiệp vụ

- Chữ ký cần lưu đúng với văn bản đã ký.
- Nếu chữ ký bị sửa, xác thực sẽ thất bại.
- Nên lưu thêm thông tin thuật toán để dễ kiểm tra.

5.10. Chức năng xác thực chữ ký

Mục đích

Kiểm tra chữ ký số có hợp lệ với nội dung văn bản và khóa công khai hay không.

Đầu vào

Người dùng cung cấp:

- Văn bản cần xác thực.
- Chữ ký số.
- Khóa công khai.

Xử lý nghiệp vụ

Hệ thống:

1. Kiểm tra văn bản có rỗng không.
2. Kiểm tra chữ ký có rỗng không.
3. Kiểm tra khóa công khai có tồn tại không.
4. Băm lại văn bản bằng SHA-256.
5. Giải mã chữ ký từ Base64.
6. Dùng khóa công khai RSA để xác thực chữ ký.
7. Trả về kết quả xác thực.

Đầu ra

Nếu hợp lệ:

Xác thực thành công.
Chữ ký hợp lệ.
Văn bản chưa bị sửa đổi.

Nếu không hợp lệ:

Xác thực thất bại.
Văn bản có thể đã bị sửa đổi, chữ ký bị sai hoặc dùng sai khóa công khai.

Quy tắc nghiệp vụ

- Chữ ký chỉ hợp lệ nếu đúng văn bản, đúng chữ ký và đúng khóa công khai.
- Nếu văn bản bị sửa một ký tự, xác thực phải thất bại.
- Nếu dùng sai khóa công khai, xác thực phải thất bại.
- Nếu chữ ký bị thay đổi, xác thực phải thất bại.

5.11. Chức năng kiểm tra văn bản bị sửa đổi

Mục đích

Phát hiện văn bản đã bị thay đổi sau khi ký.

Nguyên lý

Hệ thống không cần biết chính xác người dùng đã sửa chỗ nào. Hệ thống chỉ cần xác định rằng nội dung hiện tại không còn khớp với chữ ký ban đầu.

Các tình huống cần phát hiện

Tình huống	Kết quả
Văn bản gốc, chữ ký đúng, khóa đúng	Hợp lệ
Văn bản bị sửa một ký tự	Không hợp lệ
Văn bản bị thêm dấu cách	Không hợp lệ
Văn bản bị xóa một dòng	Không hợp lệ
Văn bản bị thay đổi dấu câu	Không hợp lệ
Chữ ký bị sửa	Không hợp lệ
Dùng sai khóa công khai	Không hợp lệ

Thông báo kết quả

Hệ thống nên thông báo rõ:

Văn bản không hợp lệ. Nội dung có thể đã bị sửa đổi sau khi ký.

Không nên chỉ ghi:

Lỗi xác thực.

Vì thông báo này không rõ ràng cho người dùng.

5.12. Chức năng hiển thị lịch sử ký

Mục đích

Cho phép người dùng xem lại các văn bản đã được ký trước đó.

Dữ liệu hiển thị

Mỗi bản ghi lịch sử ký gồm:

- Mã văn bản.
- Tên văn bản.

- Người ký.
- Mã khóa.
- Thời gian ký.
- Mã băm SHA-256.
- Đường dẫn file chữ ký.

Quy tắc nghiệp vụ

- Mỗi lần ký thành công sẽ tạo một bản ghi lịch sử.
- Không lưu lịch sử nếu quá trình ký thất bại.
- Có thể cho phép người dùng tải lại chữ ký từ lịch sử.

5.13. Chức năng hiển thị lịch sử xác thực

Mục đích

Ghi nhận các lần người dùng xác thực văn bản.

Dữ liệu lưu

Mỗi lần xác thực gồm:

- Tên văn bản.
- Thời gian xác thực.
- Khóa công khai sử dụng.
- Kết quả xác thực.
- Thông báo chi tiết.

Ví dụ:

Văn bản: hopdong.txt
Thời gian: 05/06/2026 11:00:00
Kết quả: Không hợp lệ
Lý do: Văn bản có thể đã bị sửa đổi hoặc chữ ký không đúng.

Quy tắc nghiệp vụ

- Lưu cả kết quả hợp lệ và không hợp lệ.
 - Không cần lưu nội dung đầy đủ của văn bản nếu muốn bảo vệ dữ liệu.
 - Có thể chỉ lưu mã băm để đối chiếu.
-

6. Yêu cầu phi chức năng

6.1. Yêu cầu bảo mật

- Khóa riêng tư phải được bảo vệ.
- Không hiển thị khóa riêng tư công khai nếu không cần thiết.
- Không lưu mật khẩu hoặc khóa nhạy cảm dưới dạng dễ đọc nếu triển khai nâng cao.
- Chỉ dùng thuật toán băm an toàn như SHA-256.
- Không dùng MD5 hoặc SHA-1 cho chữ ký số.
- Không cho phép ký văn bản rỗng.

6.2. Yêu cầu dễ sử dụng

- Giao diện cần đơn giản, dễ hiểu.
- Người dùng không cần hiểu sâu về RSA vẫn có thể sử dụng.
- Các nút chức năng nên rõ ràng: Tạo khóa, Ký văn bản, Xác thực.
- Thông báo lỗi phải dễ hiểu.

6.3. Yêu cầu mở rộng

Hệ thống có thể mở rộng thêm:

- Ký file PDF.
- Ký file DOCX.
- Đăng nhập người dùng.
- Mỗi người dùng có một cặp khóa riêng.
- Tích hợp mã QR chứa chữ ký.
- Thêm thời gian ký.
- Thêm chứng thư số.

7. Thiết kế module chương trình

7.1. Module KeyManager

Chức năng

Quản lý khóa RSA.

Các hàm nên có

```
create_key(key_name, owner_name)
list_keys()
load_private_key(key_id)
load_public_key(key_id)
delete_key(key_id)
```



```
export_public_key(key_id)
export_private_key(key_id)
```

Nhiệm vụ

- Sinh cặp khóa RSA.
- Lưu khóa vào thư mục.
- Đọc khóa từ file.
- Xóa khóa.
- Kiểm tra khóa tồn tại hay chưa.

7.2. Module HashService

Chức năng

Tạo mã băm SHA-256 cho văn bản.

Các hàm nên có

```
hash_text(text)
hash_file(file_path)
compare_hash(hash1, hash2)
```

Nhiệm vụ

- Nhận nội dung văn bản.
- Tạo mã băm SHA-256.
- Trả về hash dạng chuỗi.
- Hỗ trợ kiểm tra hai hash có giống nhau không.

7.3. Module SignatureService

Chức năng

Ký và xác thực chữ ký số.

Các hàm nên có

```
sign_text(text, private_key)
verify_text(text, signature, public_key)
encode_signature(signature_bytes)
decode_signature(signature_base64)
```

Nhiệm vụ

- Ký văn bản bằng khóa riêng tư.
 - Xác thực văn bản bằng khóa công khai.
 - Chuyển chữ ký sang Base64.
 - Đọc chữ ký từ Base64 về bytes.
-

7.4. Module DocumentService

Chức năng

Xử lý văn bản đầu vào.

Các hàm nên có

```
read_text_file(file_path)
save_text_file(content, file_name)
validate_text(content)
normalize_text(content)
```

Nhiệm vụ

- Đọc nội dung văn bản từ file.
 - Kiểm tra văn bản có hợp lệ không.
 - Lưu văn bản nếu cần.
 - Chuẩn hóa nội dung trước khi ký.
-

7.5. Module HistoryService

Chức năng

Lưu lịch sử ký và xác thực.

Các hàm nên có

```
save_sign_history(data)
save_verify_history(data)
get_sign_history()
get_verify_history()
```

Nhiệm vụ

- Lưu thông tin mỗi lần ký.

- Lưu thông tin mỗi lần xác thực.
- Truy xuất lịch sử để hiển thị.

8. Thiết kế thư mục dự án đề xuất

Nếu làm bằng Python Flask, có thể tổ chức như sau:

```
rsa-digital-signature/
├── app.py
├── requirements.txt
├── services/
│   ├── key_manager.py
│   ├── hash_service.py
│   ├── signature_service.py
│   ├── document_service.py
│   └── history_service.py
├── storage/
│   ├── keys/
│   ├── documents/
│   ├── signatures/
│   └── histories/
├── templates/
│   ├── index.html
│   ├── keys.html
│   ├── sign.html
│   ├── verify.html
│   └── history.html
├── static/
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── main.js
└── README.md
```

9. Thiết kế màn hình giao diện

9.1. Màn hình trang chủ

Mục đích

Giới thiệu hệ thống và điều hướng đến các chức năng chính.

Thành phần giao diện

- Tên hệ thống.
 - Mô tả ngắn.
 - Nút "Tạo khóa RSA".
 - Nút "Ký văn bản".
 - Nút "Xác thực văn bản".
 - Nút "Lịch sử".
-

9.2. Màn hình quản lý khóa

Chức năng

- Tạo khóa RSA.
- Xem danh sách khóa.
- Xem khóa công khai.
- Tải khóa công khai.
- Xóa khóa.

Thành phần giao diện

Ô nhập tên khóa
Ô nhập tên chủ sở hữu
Nút tạo khóa
Bảng danh sách khóa
Nút xem khóa công khai
Nút xóa khóa

9.3. Màn hình ký văn bản

Chức năng

- Nhập văn bản.
- Chọn khóa riêng tư.
- Ký văn bản.
- Hiển thị mã băm.

- Hiển thị chữ ký.
- Tải chữ ký.

Thành phần giao diện

Ô nhập nội dung văn bản
Danh sách chọn khóa
Nút ký văn bản
Ô hiển thị SHA-256
Ô hiển thị chữ ký số
Nút tải chữ ký

9.4. Màn hình xác thực văn bản

Chức năng

- Nhập văn bản cần kiểm tra.
- Nhập chữ ký số.
- Chọn khóa công khai.
- Xác thực.
- Hiển thị kết quả.

Thành phần giao diện

Ô nhập văn bản cần xác thực
Ô nhập chữ ký số
Danh sách chọn khóa công khai
Nút xác thực
Khu vực hiển thị kết quả

Kết quả hiển thị

Nếu hợp lệ:

Chữ ký hợp lệ.
Văn bản chưa bị sửa đổi.

Nếu không hợp lệ:

Chữ ký không hợp lệ.
Văn bản có thể đã bị sửa đổi hoặc dùng sai khóa.

10. Thiết kế dữ liệu lưu trữ

10.1. Cách lưu bằng file

Phù hợp cho bản demo sinh viên.

Thư mục keys

Lưu khóa RSA.

```
keys/  
└─ Key_Minh/  
   ├── private_key.pem  
   ├── public_key.pem  
   └─ metadata.json
```

Thư mục documents

Lưu văn bản đã ký.

```
documents/  
└─ document_001.txt
```

Thư mục signatures

Lưu chữ ký số.

```
signatures/  
└─ document_001_signature.json
```

Thư mục histories

Lưu lịch sử ký và xác thực.

```
histories/  
└─ sign_history.json  
└─ verify_history.json
```

10.2. Cách lưu bằng database

Nếu làm nâng cao, có thể thiết kế các bảng sau.

Bảng users

Trường	Kiểu dữ liệu	Ý nghĩa
id	int	Mã người dùng
full_name	varchar	Họ tên
email	varchar	Email
password	varchar	Mật khẩu đã mã hóa
created_at	datetime	Ngày tạo

Bảng rsa_keys

Trường	Kiểu dữ liệu	Ý nghĩa
id	int	Mã khóa
key_name	varchar	Tên khóa
owner_name	varchar	Chủ sở hữu
public_key_path	varchar	Đường dẫn khóa công khai
private_key_path	varchar	Đường dẫn khóa riêng tư
key_size	int	Độ dài khóa
created_at	datetime	Ngày tạo

Bảng documents

Trường	Kiểu dữ liệu	Ý nghĩa
id	int	Mã văn bản
title	varchar	Tên văn bản
content	text	Nội dung văn bản
hash_value	varchar	Mã băm SHA-256
created_at	datetime	Ngày tạo

Bảng signatures

Trường	Kiểu dữ liệu	Ý nghĩa
id	int	Mã chữ ký
document_id	int	Mã văn bản
key_id	int	Mã khóa

Trường	Kiểu dữ liệu	Ý nghĩa
signature_value	text	Chữ ký số
algorithm	varchar	Thuật toán ký
signed_at	datetime	Thời gian ký

Bảng verification_logs

Trường	Kiểu dữ liệu	Ý nghĩa
id	int	Mã log
document_name	varchar	Tên văn bản
key_id	int	Khóa dùng xác thực
result	varchar	Kết quả
message	text	Thông báo
verified_at	datetime	Thời gian xác thực

11. Thiết kế API nếu làm web backend

Nếu làm backend dạng API, có thể thiết kế như sau.

11.1. API tạo khóa

POST /api/keys

Request

```
{
  "keyName": "Key_Minh",
  "ownerName": "Nguyen Van Minh",
  "keySize": 2048
}
```

Response

```
{
  "success": true,
  "message": "Tạo khóa RSA thành công",
  "data": {
    "keyId": "Key_Minh",
  }
}
```



```
    "ownerName": "Nguyen Van Minh",
    "publicKeyPath": "keys/Key_Minh/public_key.pem"
  }
}
```

11.2. API lấy danh sách khóa

```
GET /api/keys
```

Response

```
{
  "success": true,
  "data": [
    {
      "keyId": "Key_Minh",
      "ownerName": "Nguyen Van Minh",
      "createdAt": "2026-06-05 10:30:00"
    }
  ]
}
```

11.3. API ký văn bản

```
POST /api/sign
```

Request

```
{
  "keyId": "Key_Minh",
  "content": "Tôi đồng ý với nội dung văn bản này."
}
```

Response

```
{
  "success": true,
  "message": "Ký văn bản thành công",
  "data": {
    "hash": "a3f5c9e8...",
  }
}
```

```
    "signature": "Base64SignatureHere",
    "algorithm": "RSA-SHA256",
    "signedAt": "2026-06-05 10:30:00"
  }
}
```

11.4. API xác thực văn bản

`POST /api/verify`

Request

```
{
  "keyId": "Key_Minh",
  "content": "Tôi đồng ý với nội dung văn bản này.",
  "signature": "Base64SignatureHere"
}
```

Response hợp lệ

```
{
  "success": true,
  "valid": true,
  "message": "Chữ ký hợp lệ. Văn bản chưa bị sửa đổi."
}
```

Response không hợp lệ

```
{
  "success": true,
  "valid": false,
  "message":
    "Chữ ký không hợp lệ. Văn bản có thể đã bị sửa đổi hoặc dùng sai khóa."
}
```

12. Danh sách lỗi cần xử lý

12.1. Lỗi khi tạo khóa

Mã lỗi	Trường hợp
KEY_NAME_EMPTY	Tên khóa rỗng
OWNER_NAME_EMPTY	Tên chủ sở hữu rỗng
KEY_ALREADY_EXISTS	Khóa đã tồn tại
KEY_CREATE_FAILED	Tạo khóa thất bại

12.2. Lỗi khi ký văn bản

Mã lỗi	Trường hợp
TEXT_EMPTY	Văn bản rỗng
PRIVATE_KEY_NOT_FOUND	Không tìm thấy khóa riêng tư
PRIVATE_KEY_INVALID	Khóa riêng tư không hợp lệ
SIGN_FAILED	Ký văn bản thất bại

12.3. Lỗi khi xác thực

Mã lỗi	Trường hợp
TEXT_EMPTY	Văn bản rỗng
SIGNATURE_EMPTY	Chữ ký rỗng
PUBLIC_KEY_NOT_FOUND	Không tìm thấy khóa công khai
PUBLIC_KEY_INVALID	Khóa công khai không hợp lệ
SIGNATURE_INVALID	Chữ ký không hợp lệ
VERIFY_FAILED	Xác thực thất bại

13. Các trường hợp kiểm thử

13.1. Kiểm thử tạo khóa

STT	Trường hợp	Kết quả mong muốn
1	Nhập đầy đủ tên khóa và chủ sở hữu	Tạo khóa thành công
2	Bỏ trống tên khóa	Báo lỗi

STT	Trường hợp	Kết quả mong muốn
3	Bỏ trống tên chủ sở hữu	Báo lỗi
4	Tạo khóa trùng tên	Báo lỗi
5	Xóa khóa đã tồn tại	Xóa thành công

13.2. Kiểm thử ký văn bản

STT	Trường hợp	Kết quả mong muốn
1	Văn bản hợp lệ, khóa hợp lệ	Ký thành công
2	Văn bản rỗng	Báo lỗi
3	Không chọn khóa riêng tư	Báo lỗi
4	Khóa riêng tư bị sai định dạng	Báo lỗi
5	Ký văn bản nhiều dòng	Ký thành công

13.3. Kiểm thử xác thực

STT	Trường hợp	Kết quả mong muốn
1	Văn bản gốc, chữ ký đúng, khóa đúng	Hợp lệ
2	Văn bản bị sửa một ký tự	Không hợp lệ
3	Văn bản bị thêm dấu cách	Không hợp lệ
4	Chữ ký bị sửa	Không hợp lệ
5	Dùng sai khóa công khai	Không hợp lệ
6	Chữ ký rỗng	Báo lỗi
7	Không chọn khóa công khai	Báo lỗi

14. Luồng demo đề tài

14.1. Demo trường hợp hợp lệ

Bước 1: Tạo khóa RSA.

Tên khóa: Key_Minh
Chủ sở hữu: Nguyễn Văn Minh

Bước 2: Nhập văn bản.

Tôi đồng ý với nội dung văn bản này.

Bước 3: Ký văn bản bằng khóa riêng tư.

Bước 4: Hệ thống sinh chữ ký số.

Bước 5: Dùng văn bản gốc, chữ ký số và khóa công khai để xác thực.

Kết quả:

Chữ ký hợp lệ. Văn bản chưa bị sửa đổi.

14.2. Demo trường hợp văn bản bị sửa

Bước 1: Dùng văn bản đã ký trước đó.

Tôi đồng ý với nội dung văn bản này.

Bước 2: Sửa văn bản thành:

Tôi không đồng ý với nội dung văn bản này.

Bước 3: Dùng chữ ký cũ để xác thực văn bản mới.

Kết quả:

Chữ ký không hợp lệ. Văn bản có thể đã bị sửa đổi.

14.3. Demo trường hợp dùng sai khóa

Bước 1: Tạo hai cặp khóa:

Key_Minh
Key_Test

Bước 2: Ký văn bản bằng Key_Minh.

Bước 3: Xác thực bằng khóa công khai của Key_Test.

Kết quả:

Chữ ký không hợp lệ. Có thể đã dùng sai khóa công khai.

15. Công nghệ đề xuất

15.1. Hướng đơn giản

Phù hợp để làm nhanh, dễ demo:

Python Flask + HTML/CSS + cryptography

Ưu điểm:

- Dễ code.
- Dễ hiểu thuật toán.
- Dễ demo trên trình duyệt.
- Phù hợp với báo cáo học phần an ninh mạng.

15.2. Hướng nâng cao

Nếu muốn làm giống một dự án web hoàn chỉnh:

Spring Boot + React + MySQL

Ưu điểm:

- Có backend rõ ràng.
- Có database.
- Có thể quản lý người dùng.
- Phù hợp nếu muốn phát triển thành hệ thống lớn hơn.

Nhược điểm:

- Code nhiều hơn.
- Cần xử lý bảo mật, API, database, frontend.

16. Công nghệ nên dùng trong bản demo

Đối với đề tài này, bản demo nên dùng:

Thành phần	Công nghệ
Ngôn ngữ	Python

Thành phần	Công nghệ
Web framework	Flask
Thư viện RSA	cryptography
Băm dữ liệu	SHA-256
Giao diện	HTML, CSS, Bootstrap
Lưu trữ	File JSON, PEM, TXT
Dữ liệu ký	Text hoặc file TXT

17. Quy trình phát triển dự án

Giai đoạn 1: Xây dựng phần lõi

Làm các chức năng:

1. Tạo khóa RSA.
2. Băm văn bản bằng SHA-256.
3. Ký văn bản bằng khóa riêng tư.
4. Xác thực bằng khóa công khai.

Giai đoạn 2: Xây dựng lưu trữ

Làm các chức năng:

1. Lưu khóa RSA.
2. Lưu chữ ký.
3. Lưu văn bản.
4. Lưu lịch sử ký và xác thực.

Giai đoạn 3: Xây dựng giao diện

Làm các màn hình:

1. Trang chủ.
2. Trang quản lý khóa.
3. Trang ký văn bản.
4. Trang xác thực.
5. Trang lịch sử.

Giai đoạn 4: Kiểm thử

Thực hiện các test case:

1. Văn bản gốc xác thực thành công.

2. Văn bản bị sửa xác thực thất bại.
3. Sai khóa công khai xác thực thất bại.
4. Sai chữ ký xác thực thất bại.
5. Văn bản rỗng báo lỗi.

Giai đoạn 5: Viết báo cáo

Báo cáo gồm:

1. Tổng quan chữ ký số.
 2. Thuật toán RSA.
 3. Hàm băm SHA-256.
 4. Phân tích thiết kế hệ thống.
 5. Cài đặt chương trình.
 6. Kiểm thử và đánh giá.
 7. Kết luận và hướng phát triển.
-

18. Kết quả mong đợi của dự án

Sau khi hoàn thành, hệ thống cần đạt được:

- Người dùng có thể tạo cặp khóa RSA.
 - Người dùng có thể ký nội dung văn bản.
 - Hệ thống sinh ra chữ ký số.
 - Người dùng có thể xác thực văn bản bằng chữ ký và khóa công khai.
 - Hệ thống phát hiện được văn bản bị sửa đổi.
 - Hệ thống thông báo rõ ràng kết quả xác thực.
 - Có dữ liệu kiểm thử chứng minh chức năng hoạt động đúng.
-

19. Hướng phát triển thêm

Sau khi hoàn thành chức năng cơ bản, có thể mở rộng:

- Ký file PDF.
 - Ký file DOCX.
 - Tạo chữ ký hình ảnh đóng dấu lên PDF.
 - Tạo mã QR chứa chữ ký.
 - Quản lý người dùng đăng nhập.
 - Mã hóa khóa riêng tư bằng mật khẩu.
 - Thêm chứng thư số.
 - Thêm timestamp để xác nhận thời điểm ký.
 - Lưu lịch sử ký trên database.
 - So sánh hai phiên bản văn bản để chỉ ra phần bị sửa.
-

20. Kết luận

Dự án “Ứng dụng chữ ký số RSA để xác thực nội dung văn bản – có kiểm tra sửa đổi” tập trung vào việc đảm bảo tính xác thực và tính toàn vẹn của văn bản điện tử.

Hệ thống sử dụng RSA để tạo và xác thực chữ ký số, kết hợp với SHA-256 để tạo mã băm đại diện cho nội dung văn bản. Khi văn bản bị sửa đổi, mã băm thay đổi, từ đó chữ ký không còn hợp lệ. Nhờ vậy, hệ thống có thể phát hiện nội dung văn bản đã bị chỉnh sửa sau khi ký.

Các chức năng quan trọng cần triển khai gồm:

1. Tạo cặp khóa RSA.
2. Quản lý khóa công khai và khóa riêng tư.
3. Băm văn bản bằng SHA-256.
4. Ký văn bản bằng khóa riêng tư.
5. Xác thực chữ ký bằng khóa công khai.
6. Kiểm tra văn bản bị sửa đổi.
7. Lưu chữ ký và lịch sử xác thực.
8. Hiển thị kết quả rõ ràng cho người dùng.

Đây là mô hình phù hợp với sinh viên năm 3 học học phần an ninh mạng, vừa thể hiện được kiến thức lý thuyết về RSA, hàm băm và chữ ký số, vừa có khả năng triển khai thành một ứng dụng thực tế.

DIGITALLY SIGNED

By: Minh

Time: 2026-06-12 18:32:20

